

Fondamentaux du web & environnement

Développement web — Classe passerelle

Semaine 1

Anne Terrier | cfpt-i — École d'informatique

Au programme aujourd'hui

1. Comment fonctionne le web ?
2. Les langages côté client
3. Les langages côté serveur
4. La stack du cours
5. Environnement de développement
6. Git — premiers pas

Objectif de la semaine

Comprendre le fonctionnement du web et installer son environnement de développement.

1. Comment fonctionne le web ?

Le modèle client / serveur



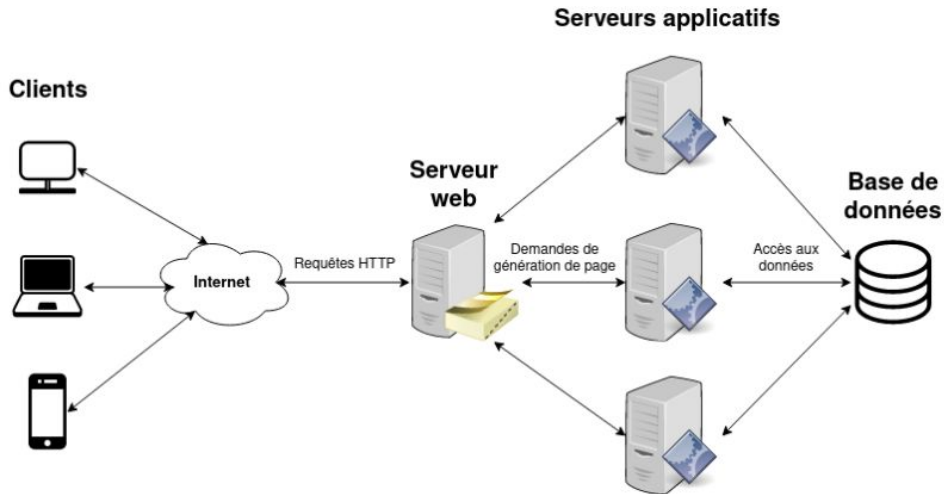
Définition — Client et serveur

Client : programme qui *demande* une ressource (le navigateur).

Serveur : programme qui *répond* en envoyant fichiers ou données.

Communication via le protocole **HTTP** (ou **HTTPS** sécurisé).

Le modèle client / serveur — en détail



Anatomie d'une URL

```
https://www.hepia.ch/formations/informatique
```

↑
Protocole

↑
Domaine

↑
Chemin

DNS — de l'adresse au nom

Un ordinateur ne comprend pas `www.hepia.ch` : il lui faut une **adresse IP** (ex. `192.0.2.34`). Le **DNS** traduit le nom de domaine en adresse IP.

Analogie — L'annuaire téléphonique

Tu connais le *nom* (le domaine), l'annuaire te donne le *numéro* (l'IP), et c'est ce numéro qui permet la communication.

Étapes simplifiées :

1. Le navigateur demande : « quelle est l'IP de `www.hepia.ch` ? »
2. Le serveur DNS répond avec l'adresse IP
3. Le navigateur envoie sa requête HTTP à cette adresse

Les codes de statut HTTP

Code	Signification	Quand ?
200	OK	Tout s'est bien passé
301	Redirection	La page a changé d'adresse
404	Not Found	La page n'existe pas
500	Server Error	Erreur côté serveur

Le fameux « 404 » que tout le monde a déjà rencontré !

Sites statiques et sites dynamiques

Définition

Statique : renvoie toujours le même contenu (fichier HTML déjà prêt).

Dynamique : génère le contenu à la demande, souvent depuis une base de données.

	Statique	Dynamique
Contenu	Identique pour tous	Généré à chaque requête
Exemple	Vitrine, portfolio	Forum, boutique, réseau social
Technique	HTML/CSS/JS seuls	Node.js + base de données

Où se situe MiniForum ?

Statique au départ (sem. 1–5), puis dynamique dès la phase 3 avec une vraie base de données.

2. Les langages côté client

HTML

Structure

Titres, paragraphes,
liens, images

Les murs

CSS

Présentation

Couleurs, polices,
mise en page

La peinture

JavaScript

Comportement

Interactions, ap-
pels serveur

L'électricité

Qui exécute quoi ?

HTML, CSS et JavaScript s'exécutent dans le **navigateur** (côté client).

Un premier exemple — la structure (HTML)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>MiniForum</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Bienvenue sur MiniForum</h1>
  <button id="btn">Cliquez-moi</button>
  <script src="app.js"></script>
</body>
</html>
```

Un premier exemple — présentation & comportement

style.css

```
button {  
  background-color: #1D3557;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

app.js

```
const btn =  
  document.getElementById('btn');  
  
btn.addEventListener('click',  
  () => {  
    alert('Bonjour depuis JS !');  
  });
```

HTML **structure**, CSS **habille**, JavaScript **anime**.

3. Les langages côté serveur

Côté serveur : un choix de langages

Dans ce cours

Node.js s'exécute sur le **serveur**. La base de données est aussi côté serveur, jamais accessible directement depuis le navigateur.

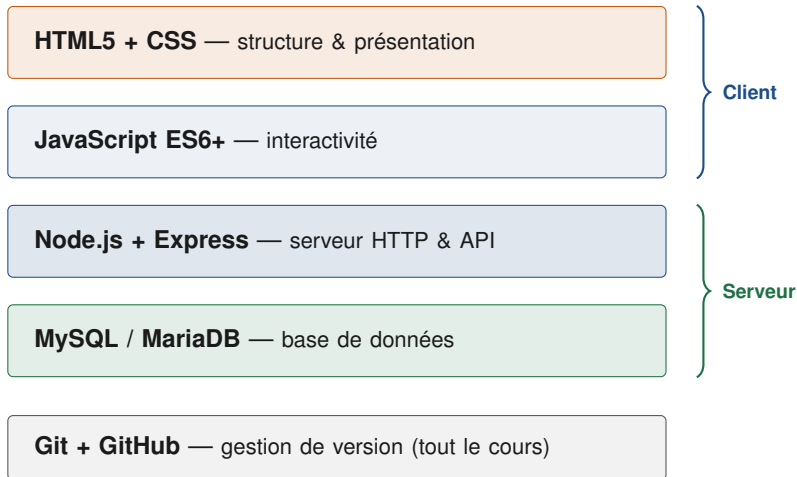
Pour info — D'autres langages, mêmes principes

Contrairement au navigateur, un serveur n'est pas limité à un seul langage : **PHP**, **Python**, **Java**, **Ruby**, **C#**... coexistent sur le web. Chacun peut jouer le même rôle que Node.js.

Le principe reste partout le même : recevoir une requête, dialoguer avec une base de données (en **SQL**), renvoyer une réponse.

4. La stack du cours

La stack de MiniForum



Une application web = trois parties

- une interface **côté client**
- un serveur **côté serveur**
- une **base de données**

La particularité de ce cours

Nous utiliserons **JavaScript des deux côtés** : dans le navigateur *et* sur le serveur (Node.js). Un seul langage à maîtriser pour toute la stack applicative.

5. Environnement de développement

À installer cette semaine

- **VS Code** — éditeur de code
- **Node.js** (LTS) — serveur JS
- **Git** — gestion de version
- Un compte **GitHub**

Extensions VS Code

- Prettier — mise en forme
- ESLint — qualité du code
- Live Server — aperçu live
- GitLens — visualiser Git

Le détail pas-à-pas se trouve dans la **fiche TP** de la semaine.

Premiers pas dans le terminal

```
pwd          # afficher le dossier courant
ls           # lister les fichiers
cd miniforum # entrer dans un dossier
cd ..        # remonter d'un niveau
mkdir css    # creer un dossier
touch app.js # creer un fichier vide
```

Ces commandes seront approfondies dans le cours **Linux / Bash**.

6. Git — premiers pas

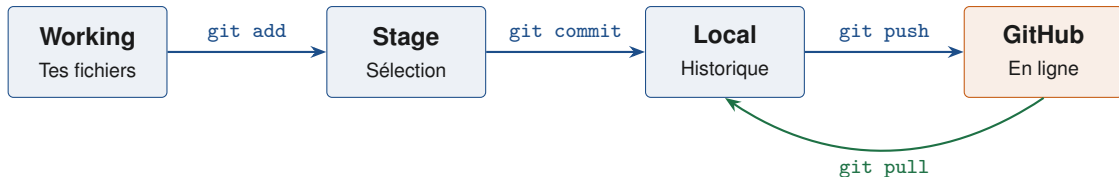
Pourquoi utiliser Git ?

- **Sauvegarder** son travail et son historique
- **Revenir en arrière** en cas d'erreur
- **Collaborer** à plusieurs sans s'écraser
- **Partager** son code (GitHub)

L'outil standard du métier

Git est utilisé par la quasi-totalité des équipes de développement dans le monde. Le maîtriser tôt est un vrai atout.

Les quatre zones de Git



Analogie — La photo de groupe

Working : tout le monde est dans la salle. **Stage** : certains se placent. **Commit** : on prend la photo (instantané). **Push** : on l'envoie sur le serveur partagé.

Commandes essentielles

```
git init                # initialiser un depot
git status              # etat des fichiers
git add .               # tout ajouter au stage
git commit -m "message" # enregistrer un commit
git push origin main    # envoyer sur GitHub
git pull                # recuperer les changements
```

Le bon réflexe

Commit **souvent**, avec des messages clairs. Un commit = une étape logique de ton travail.

Ce qu'on a vu

- Le web = client + serveur + HTTP
- URL, DNS, codes de statut
- Statique vs dynamique
- HTML / CSS / JS côté client
- La stack MiniForum

À faire pour la semaine 2

- Installer les outils (fiche TP)
- Créer son dépôt GitHub
- Faire le quiz Moodle

Des questions ?

Prochaine étape : la fiche TP — mise en place du projet